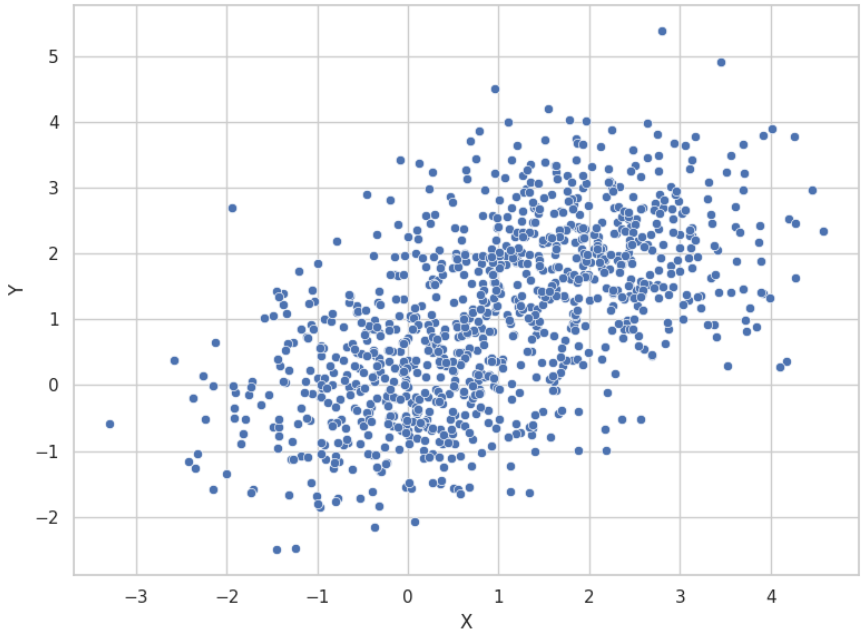
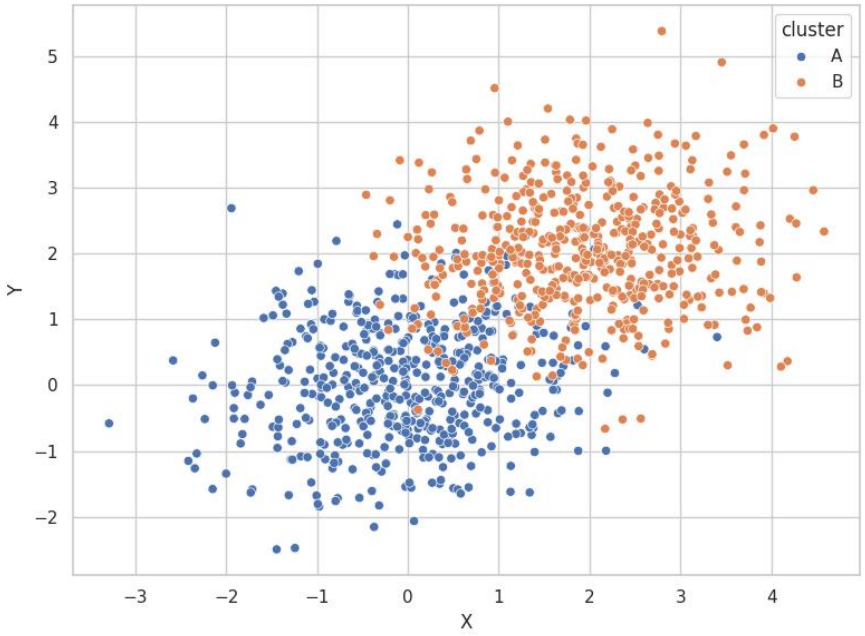
# 04

# Classification

# Regression vs classification




**REGRESSION**

Output = continuous

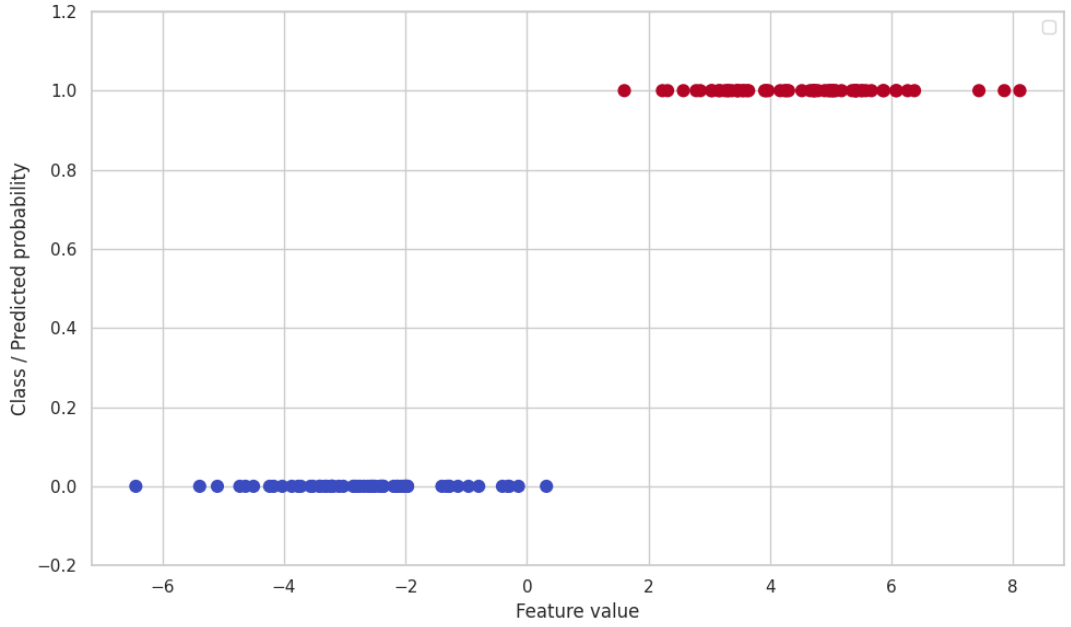
*Based on X, predict the value of Y*

**CLASSIFICATION**

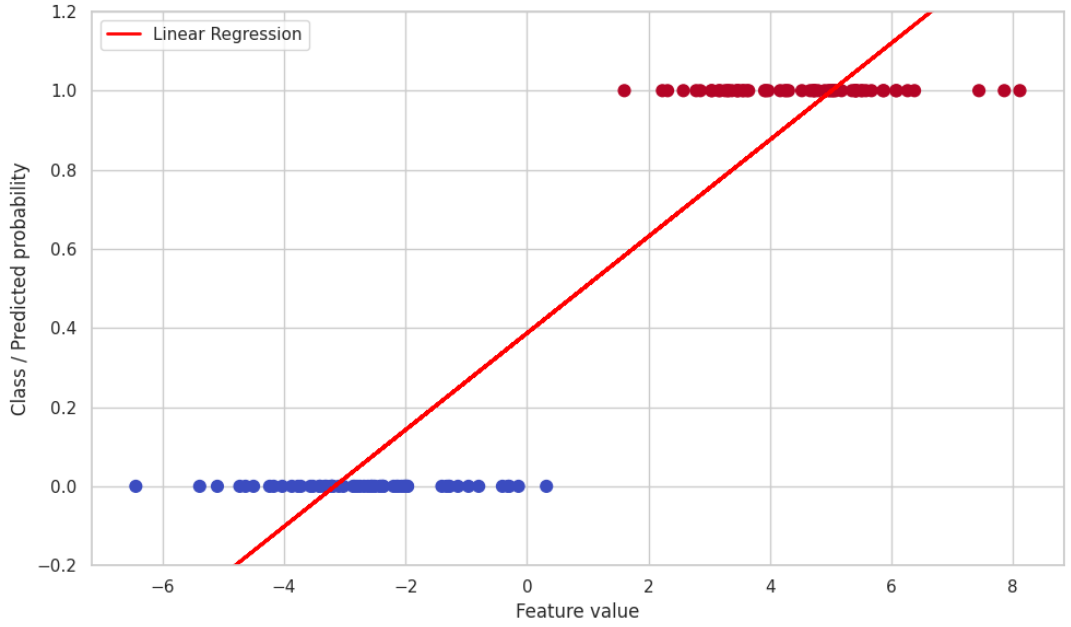
Output = discrete

*Based on X and Y, predict the class*

# Logistic regression



# Logistic regression




We need to make

**assumptions**

about the

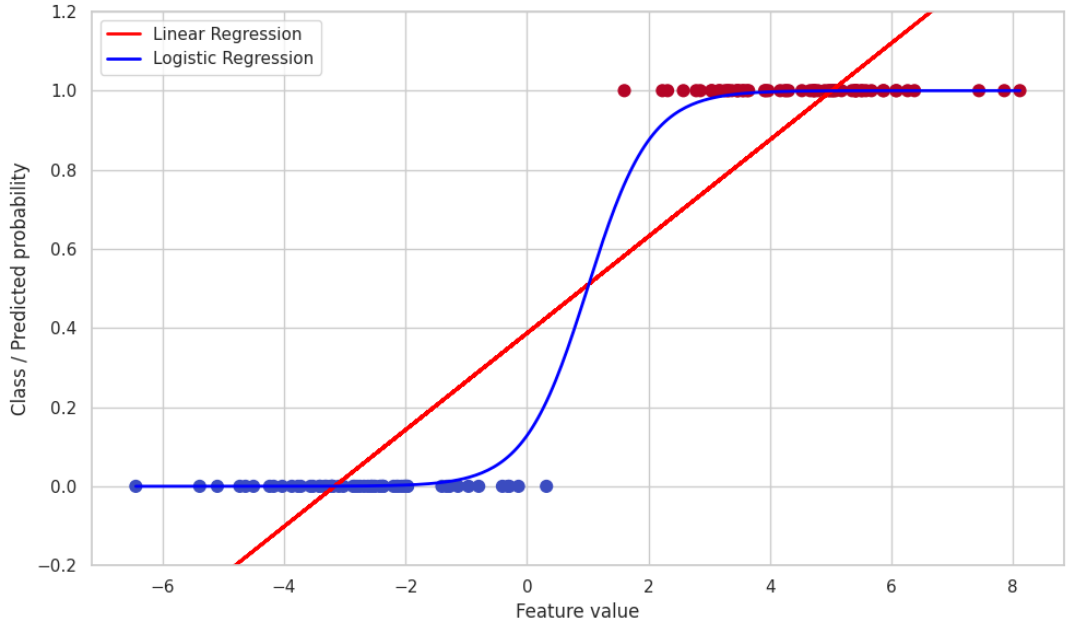
**model**

that generated the data

*Linear relationship*

*Linear model*

# Logistic regression




We need to make

**assumptions**

about the

**model**

that generated the data

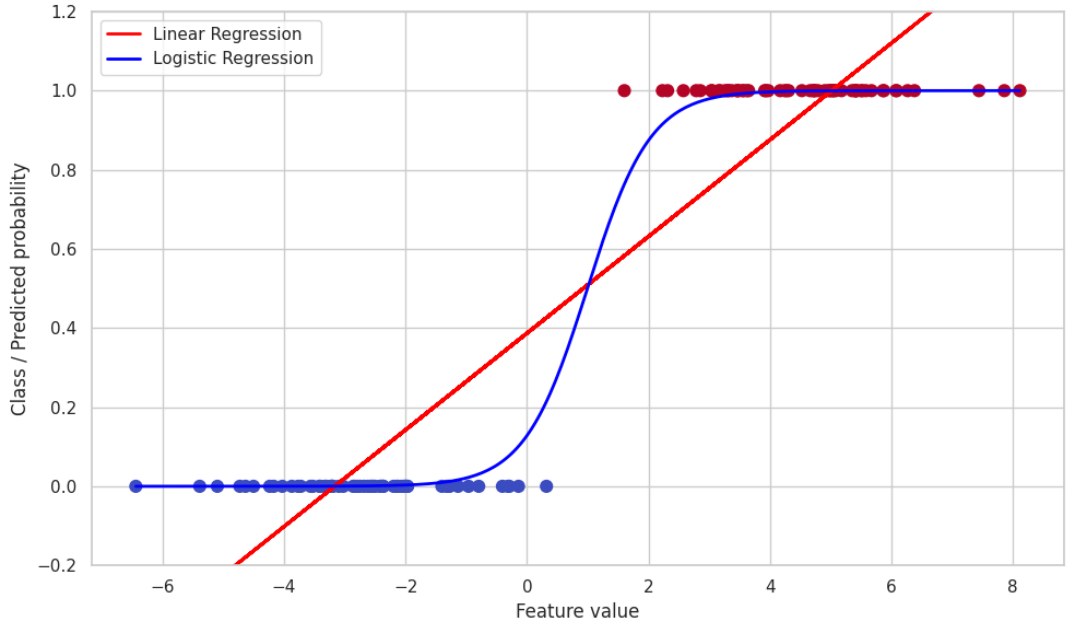
*Linearly separable*

~~*Linear relationship*~~

*Logistic model*

~~*Linear model*~~

# Logistic regression




- Sigmoid function:

$$f(x, \theta) = g(\theta_0 + \theta_1 x_1)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

- Model parameters
  - $\theta_0$ = intercept
  - $\theta_1$ = weight

# Linear regression - fitting

Fit a linear model

$$f(x,\theta) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_m x_m + b$$

to the dataset such that the cost function

$$J(\theta) = \frac{1}{2n}\sum_{i=1}^{n}(f(x,\theta) - y)^2$$

is minimal

# Logistic regression - fitting

Fit a logistic regression model

$$f(x,\theta) = g(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_m x_m + b); \qquad g(z) = \frac{1}{1+e^{-z}}$$

to the dataset such that the cost function

$$J(\theta) = -[\frac{1}{n}\sum_{i=1}^{n} y^{(i)} \log\left(f\left(x^{(i)},\theta\right)\right) + (1-y^{(i)})\log(1-f\left(x^{(i)},\theta\right)]$$

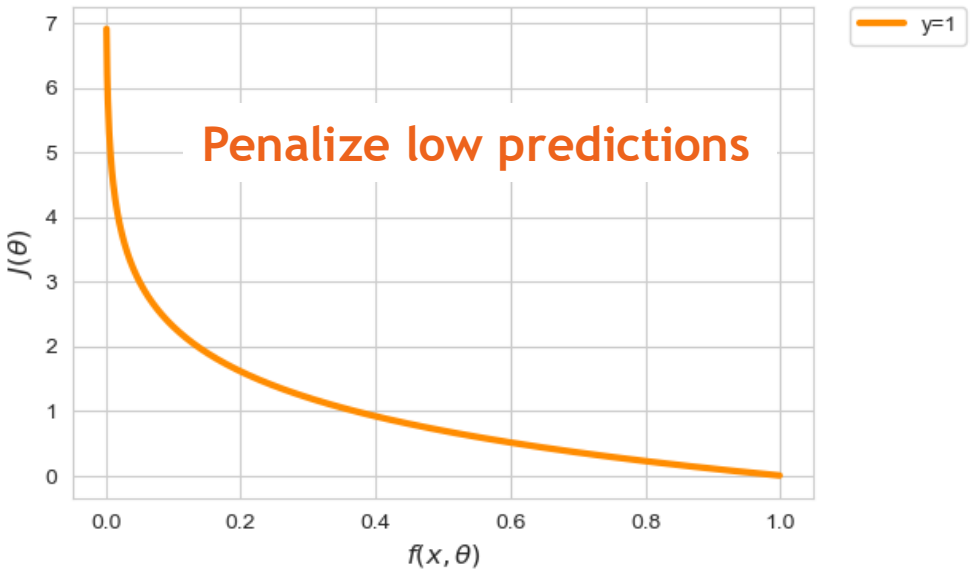
is minimal

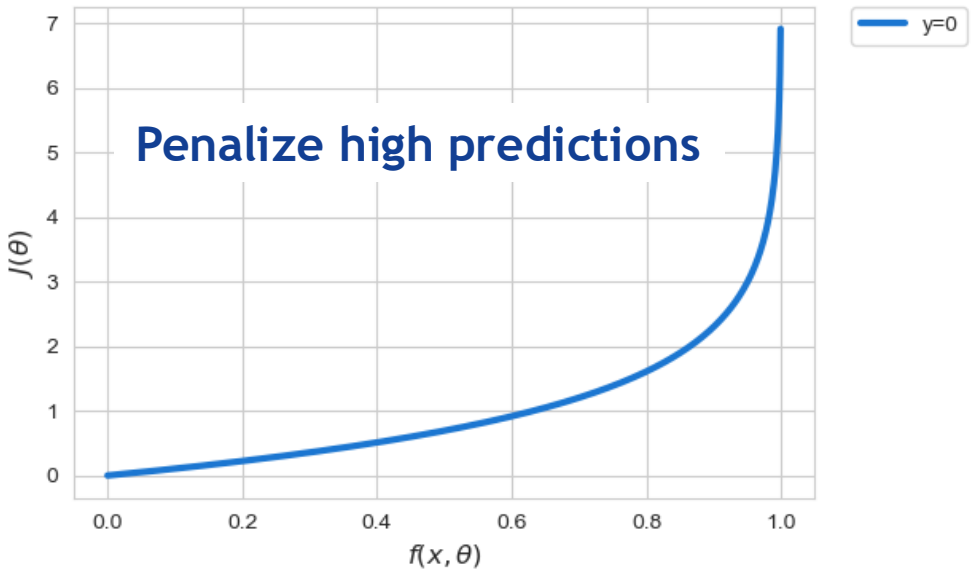
# Logistic regression - fitting

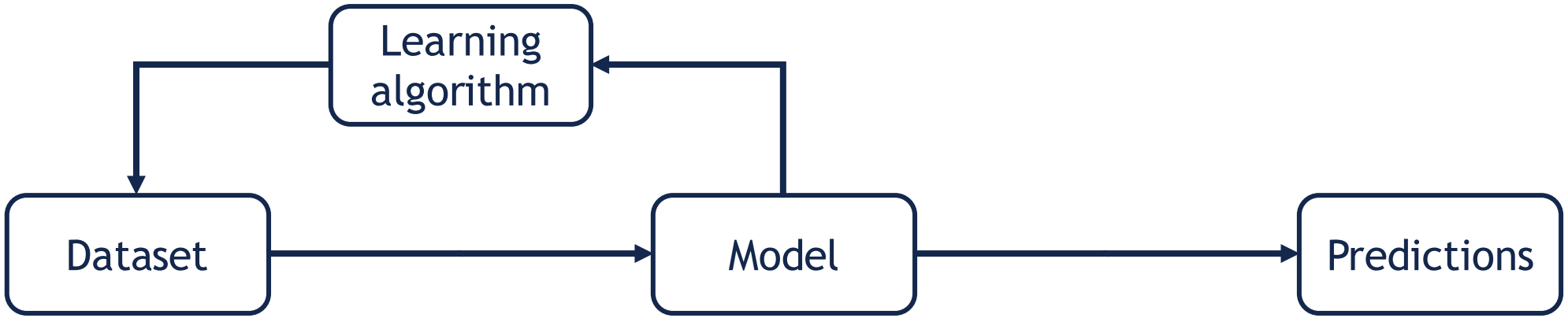
$$J(\theta) = -[\frac{1}{n}\sum_{i=1}^{n} y^{(i)} \log\left(f\left(x^{(i)},\theta\right)\right) + (1-y^{(i)})\log(1-f\left(x^{(i)},\theta\right)$$

$y^{(i)} = 1$

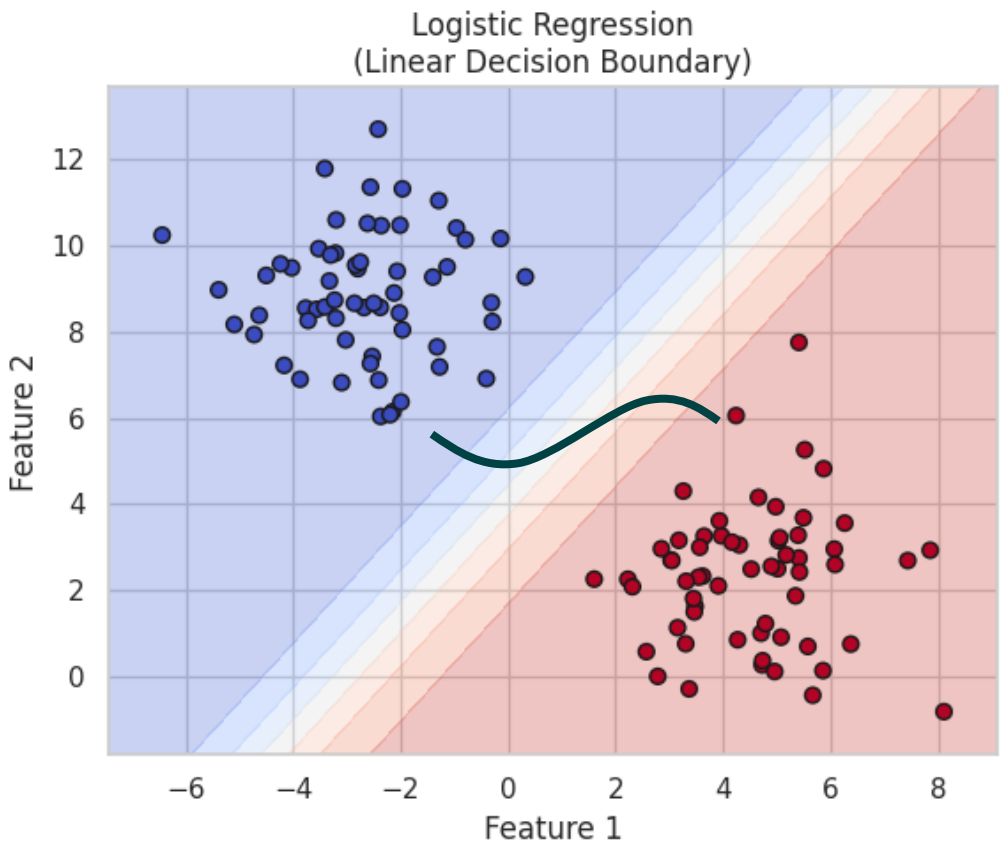
$y^{(i)} = 0$




Learning algorithm
Dataset
Model
Predictions

# Logistic regression



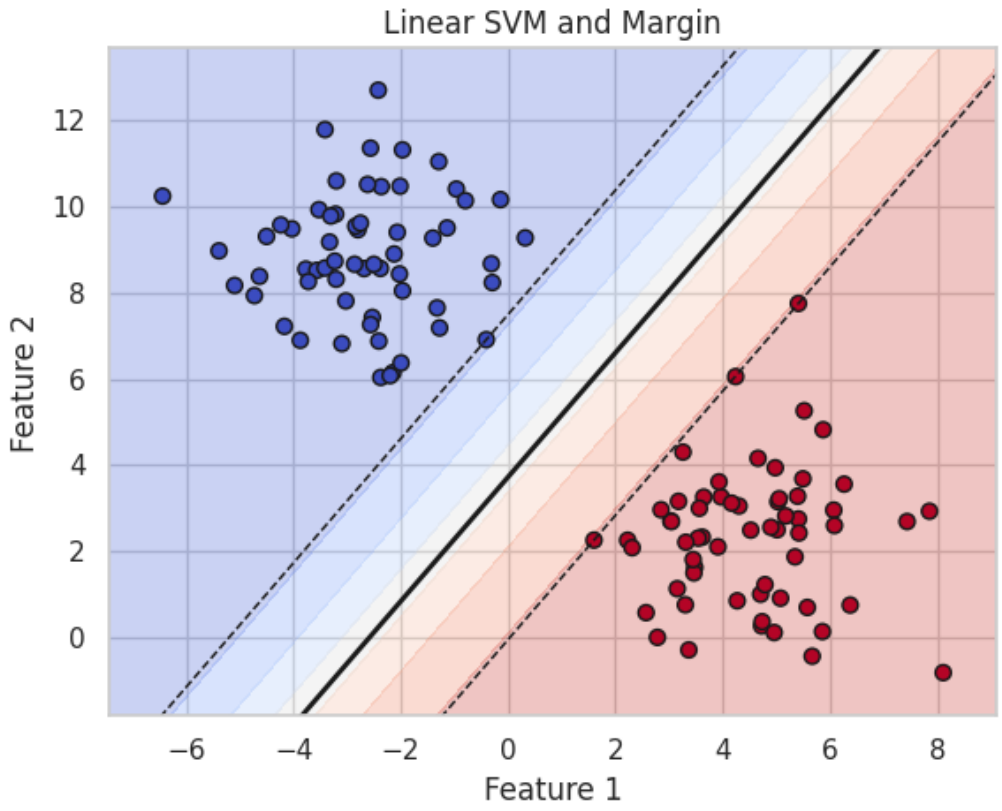

**Pro's**

- "Simple" model
  → fast and inexpensive training
- Probabilistic(-like) output
- Easily interpretable (feature weight ~ importance)

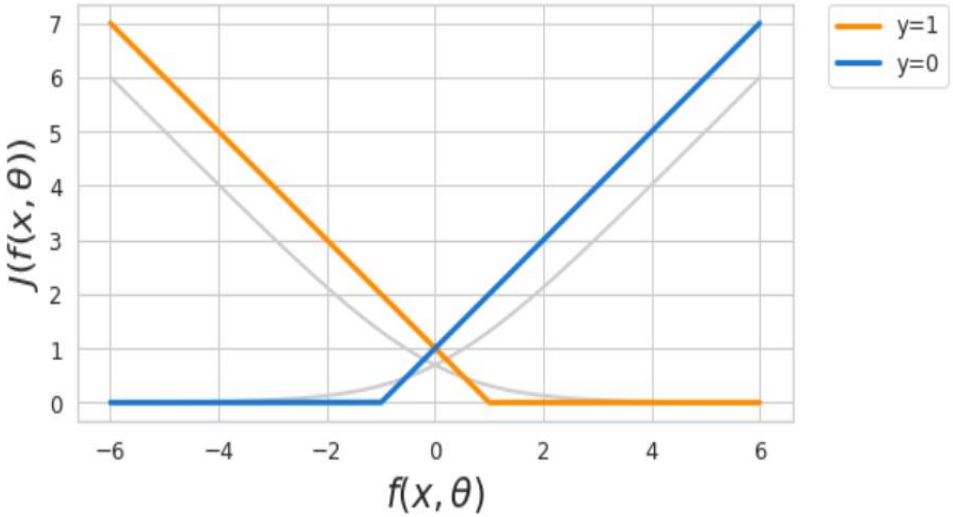
**Cons**

- Assumption of linear separability
- Sensitive to outliers
- Sensitive to correlated features

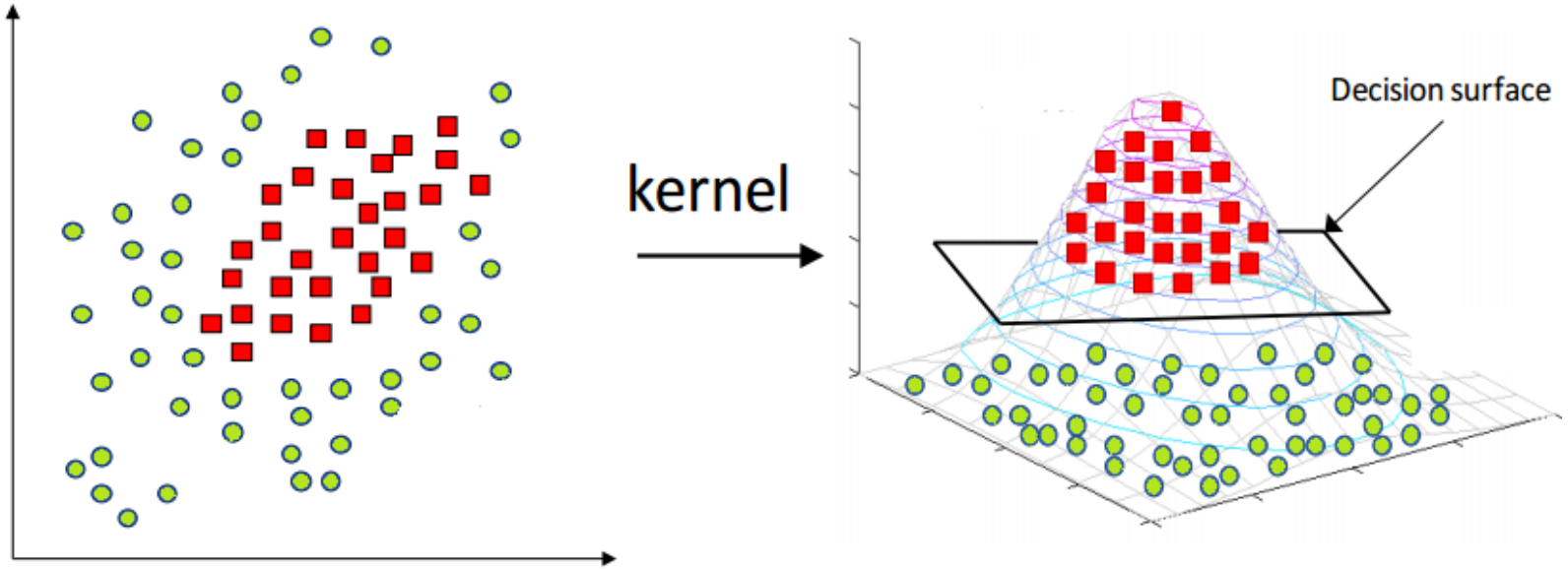
# Support Vector Machine




- Finds the optimal hyperplane to separate data points with the **maximum margin**
- The hyperplane is defined by **support vectors**, which are the data points closest to the hyperplane (→ robust to outliers)

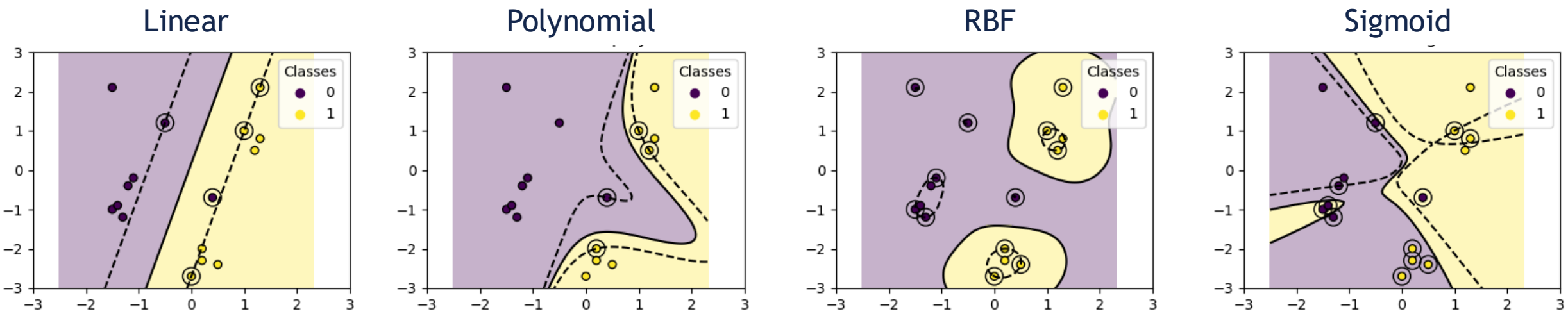


# Support Vector Machine - Kernel trick



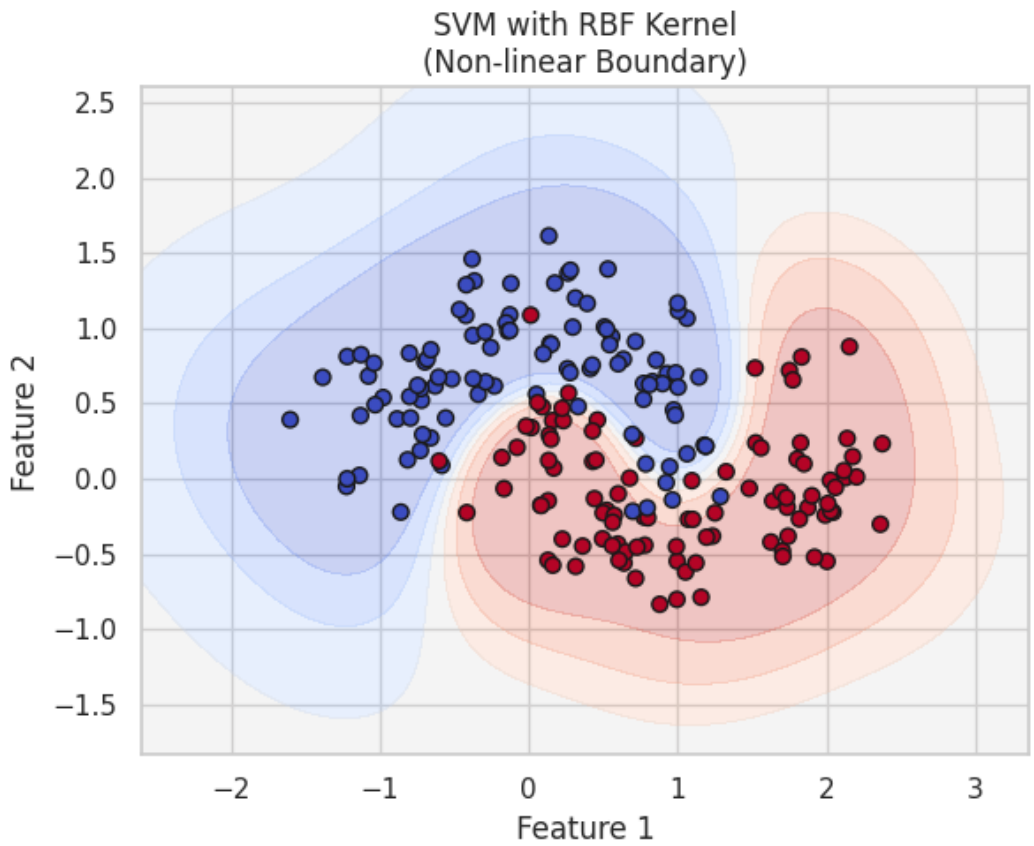

With the kernel trick we use a transformation $\phi(x)$ to map the data to a higher-dimensional representation, where the data is linearly separable.

# Support Vector Machine - Kernel trick




Scikit-learn - Plot classification boundaries with different SVM Kernels

# Support Vector Machine




## Pro's

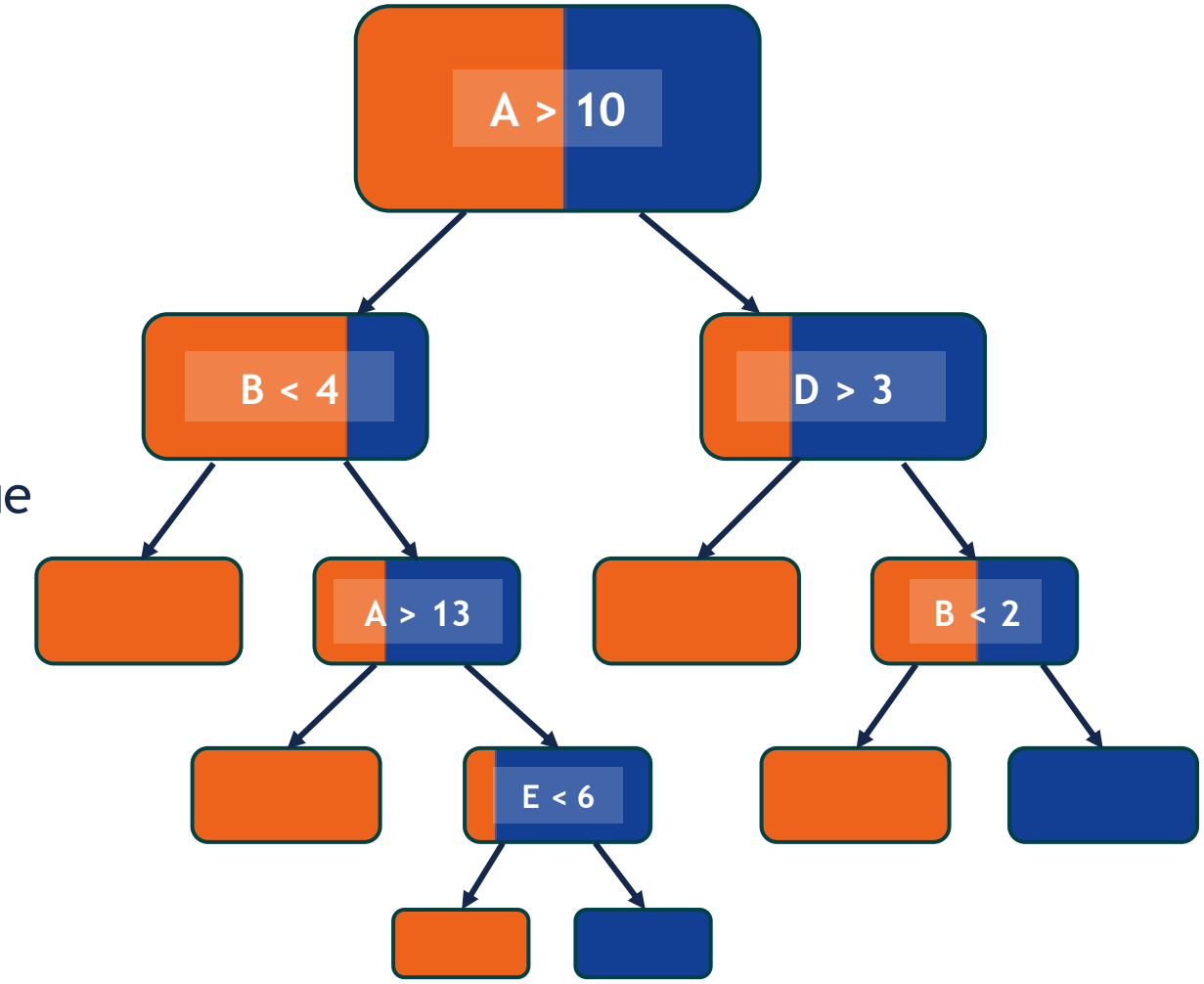
- Hyperplane is defined by support vectors (→ effective with **high number of features**, and robust to outliers)
- Maximal margin gives good generalization (if linearly separable)
- **Kernel trick** allows to model non-linear relationships

## Cons

- **Computationally expensive** for large datasets
- Hard to tune hyperparameters (regularization C, kernel type, kernel parameters)
- Not well-suited for noisy data

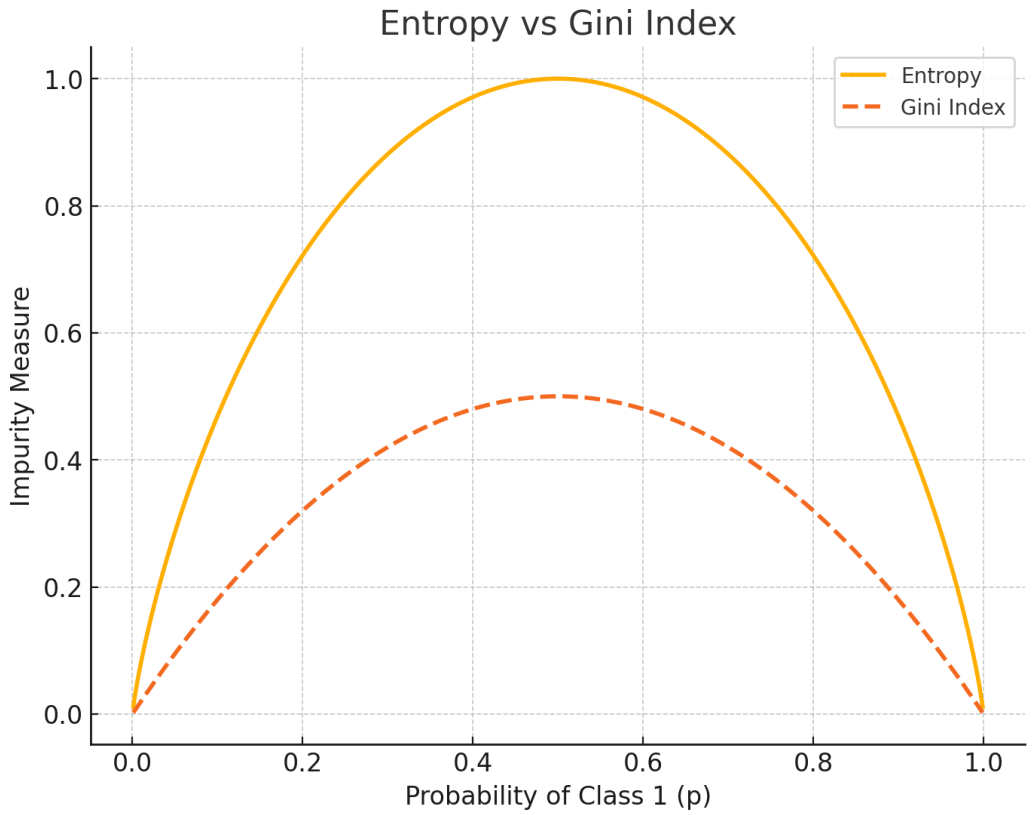
# Decision tree

- Models decisions and their possible consequences in a tree-like structure
  - Nodes = features
  - Branches = decision rules
  - Leaves = outcomes
- Recursively split data based on feature value
  - Impurity (Entropy, Gini index)
  - How *mixed* the classes are
- High interpretability, but **prone to overfitting**
  - Max tree depth
  - Pruning




VIB

# Decision tree




**Impurity measures** – how *mixed* the classes are

- Entropy = $-\sum_{i=1}^{C} p_i \log_2(p_i)$
- Gini Index = $1 - \sum_{i=1}^{C} p_i^2$

**Good split** – gives the biggest impurity decrease

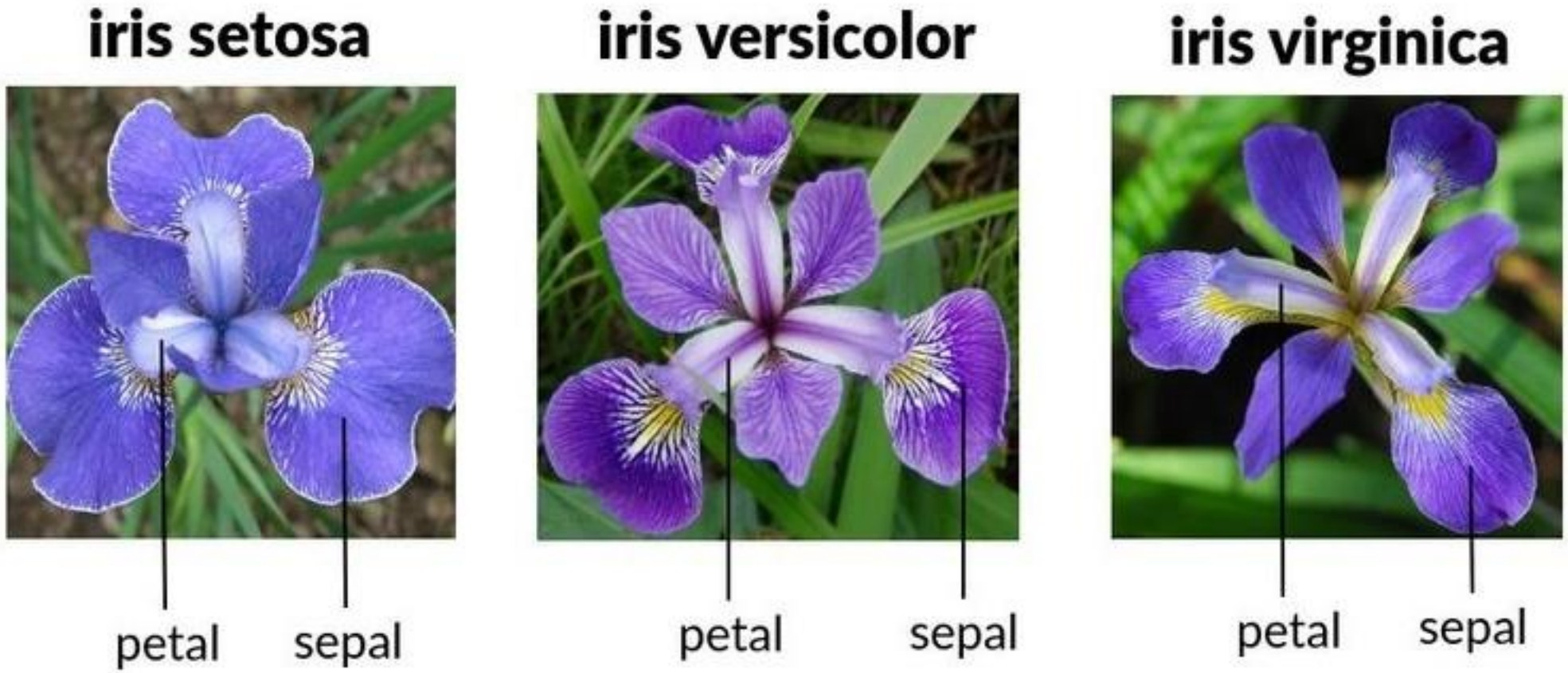
- Weighted average of the impurity of the children

*Example: 80% class A and 20% class B*

- Entropy = $-0.8 * \log_2(0.8) - 0.2 * \log_2(0.2) = 0.721$
- Gini Index = $1 - (0.8^2 + 0.2^2) = 0.32$

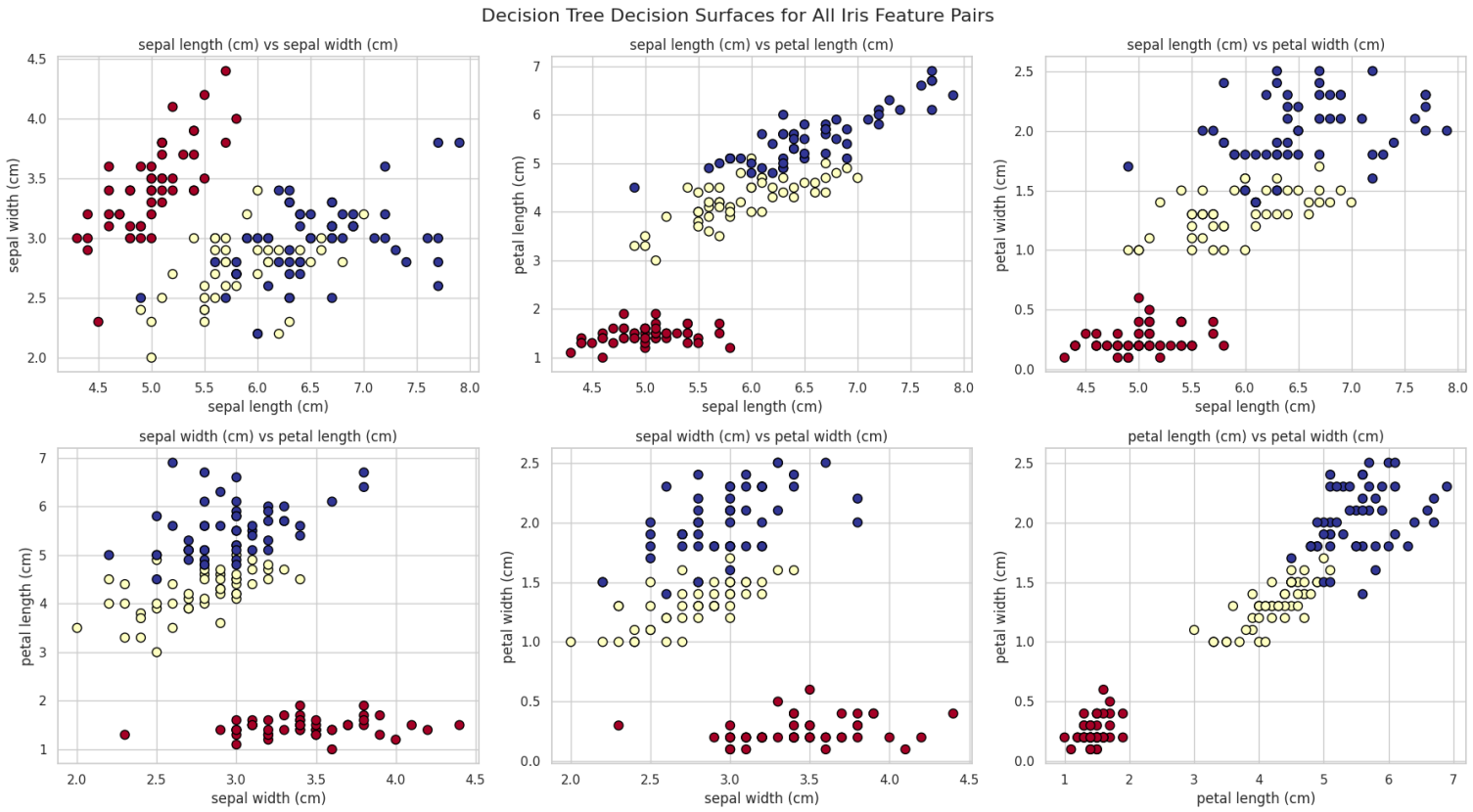
# What does the decision boundary of a decision tree look like?

# Decision tree

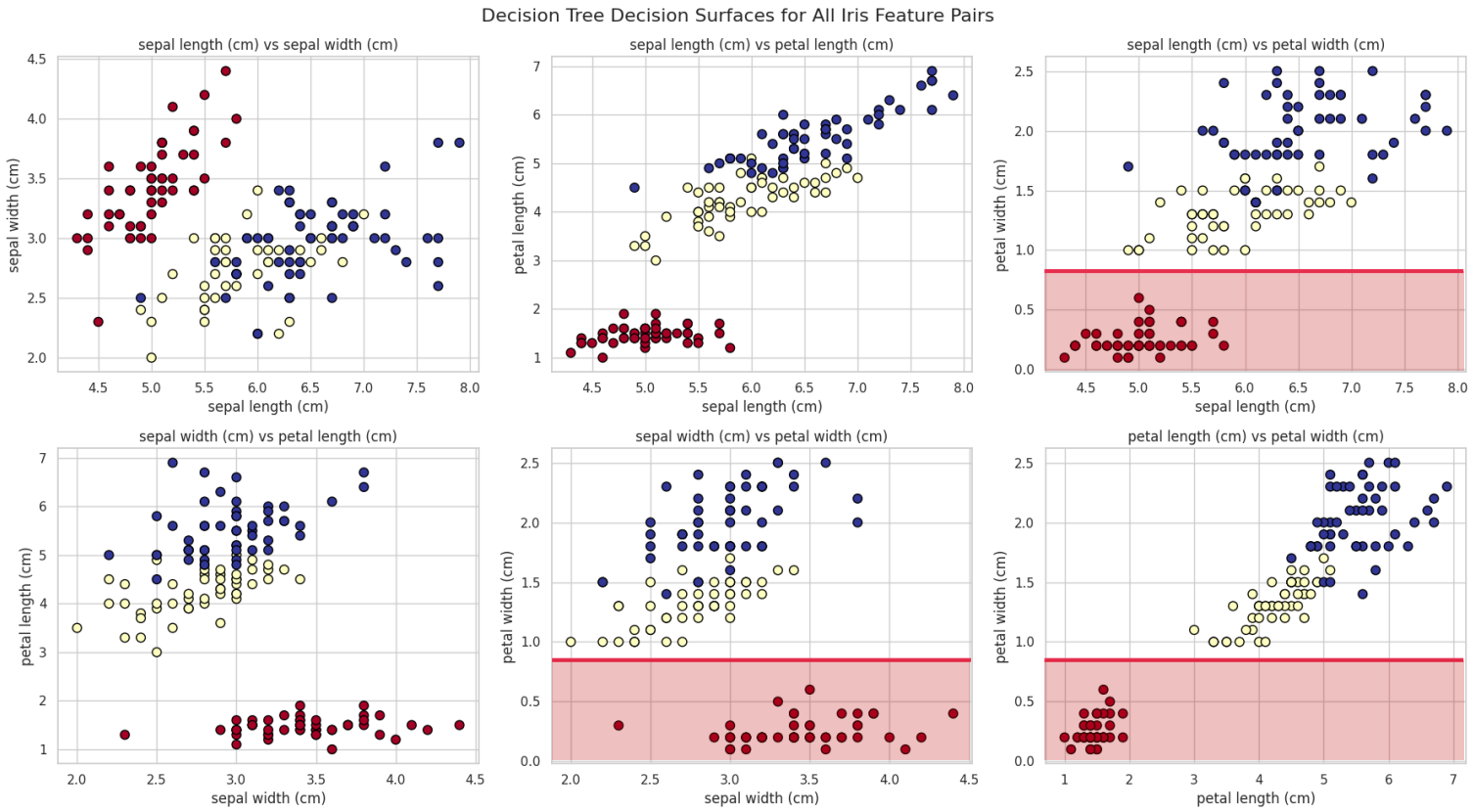


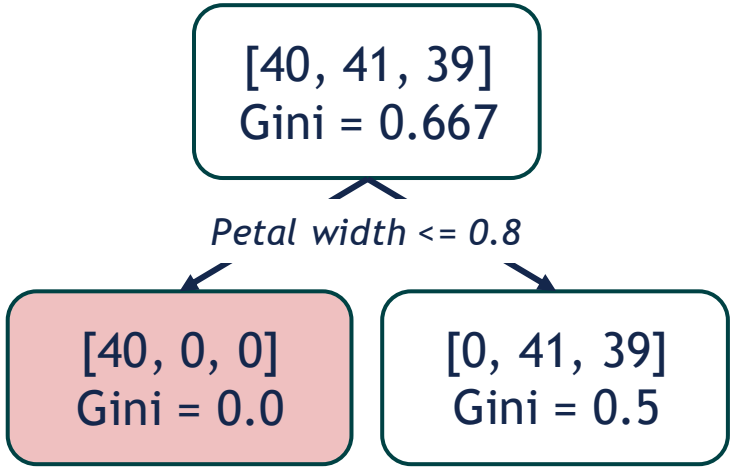
# Decision tree

[40, 41, 39]
Gini = 0.667

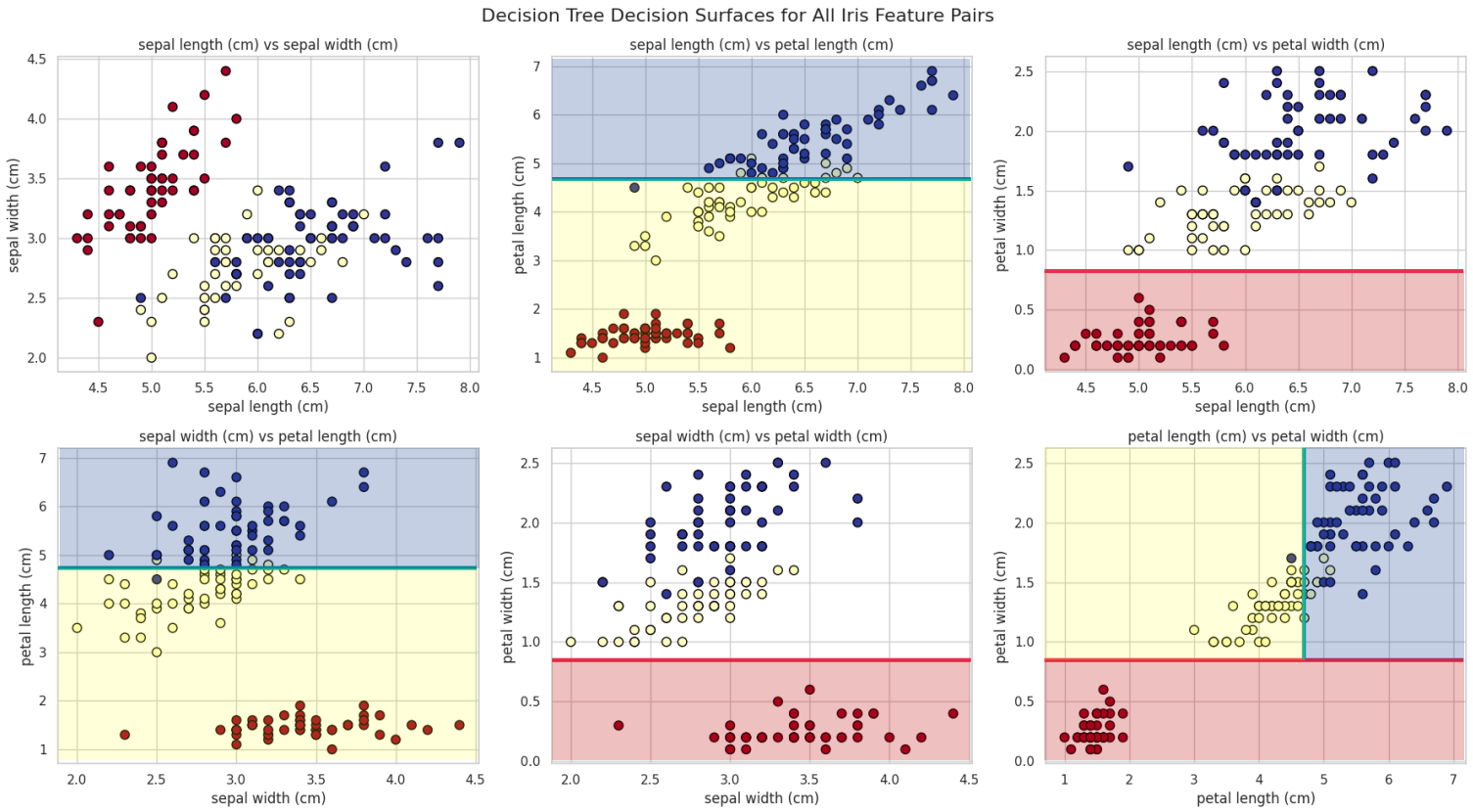


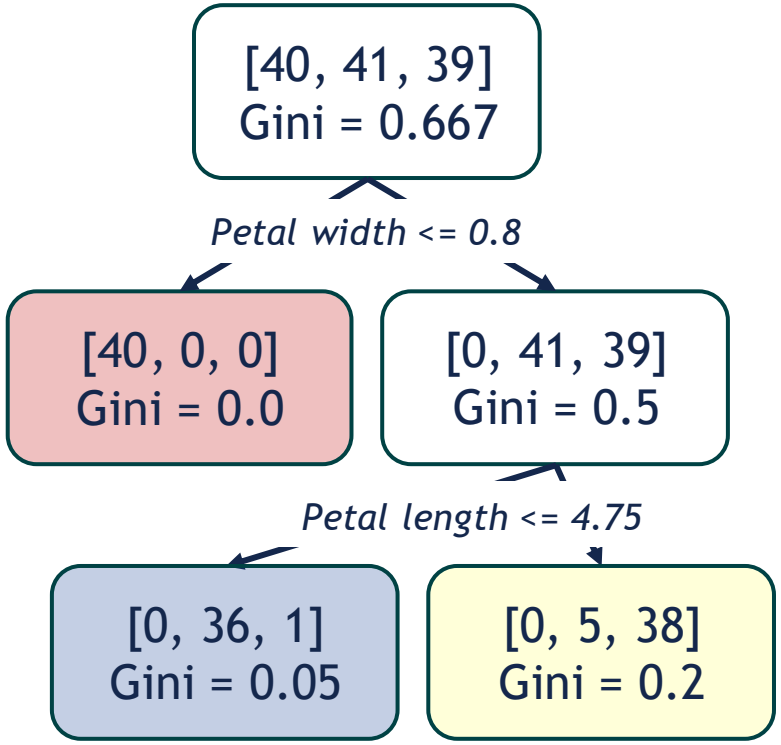
# Decision tree



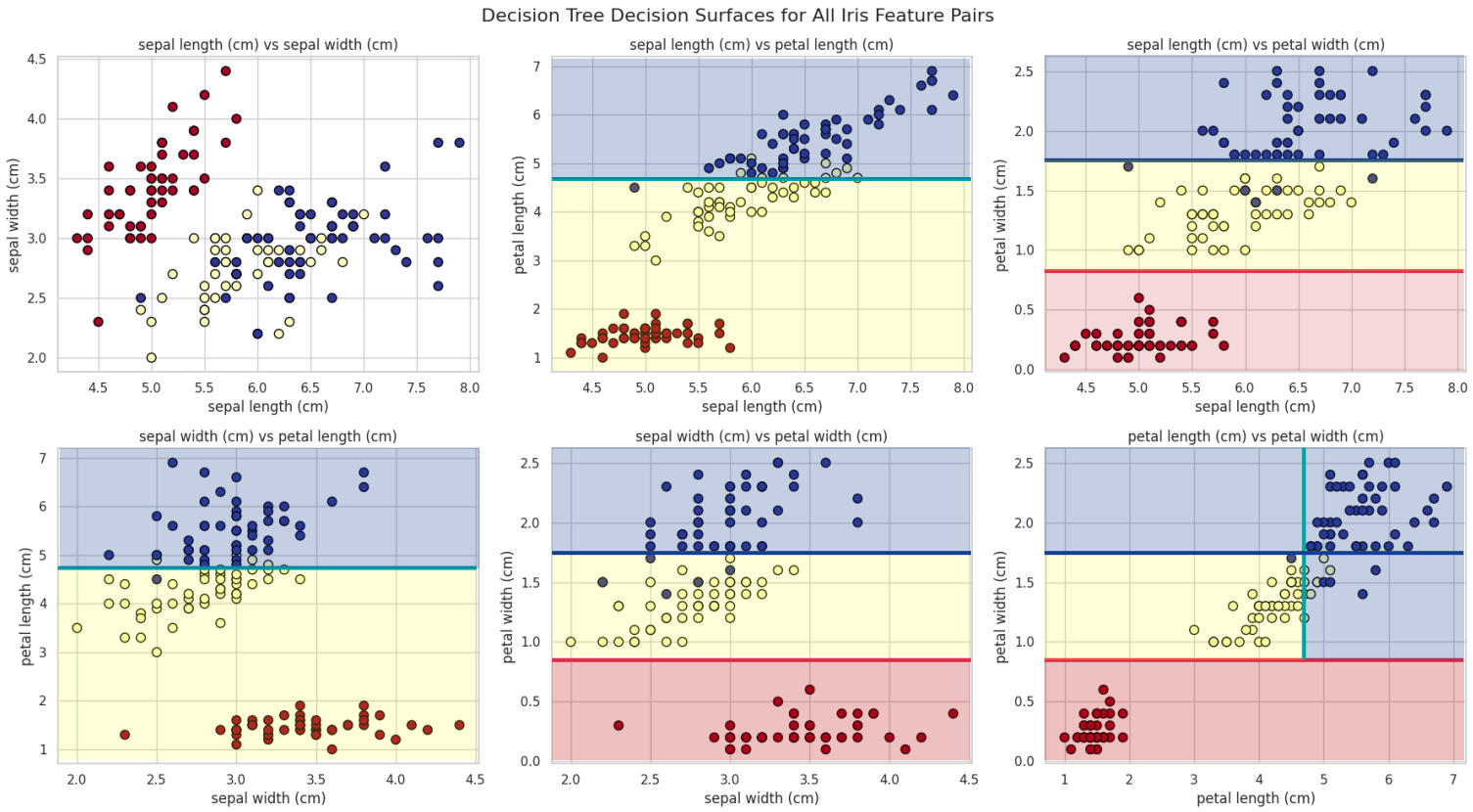



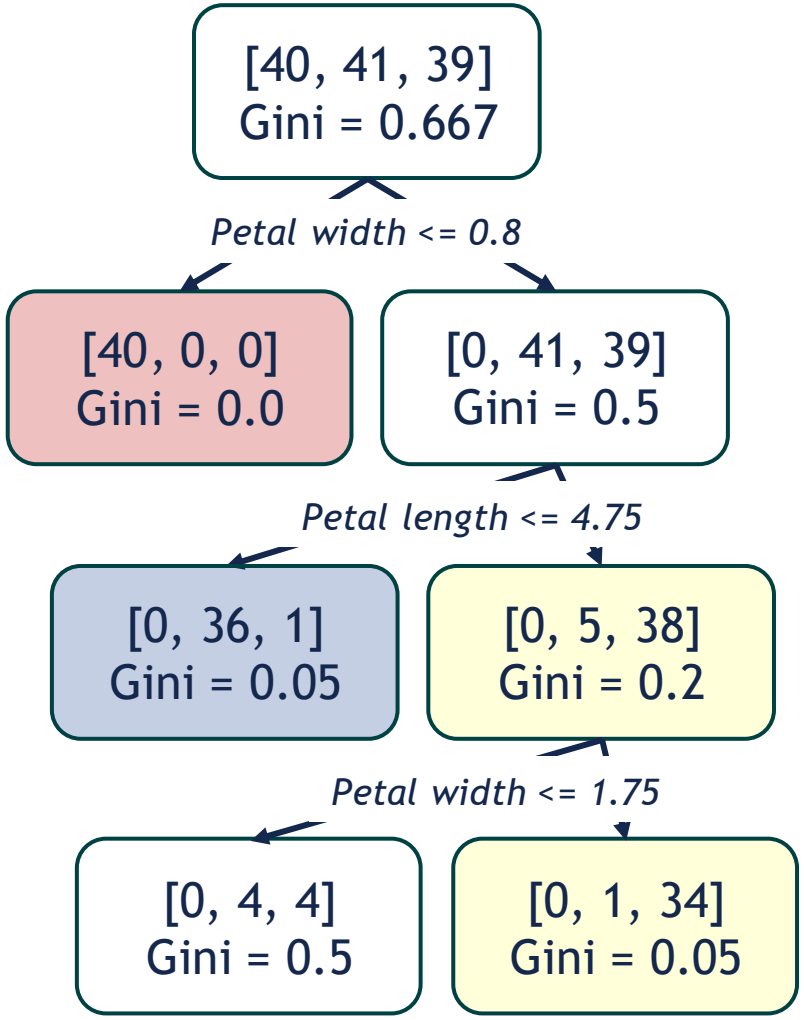
# Decision tree



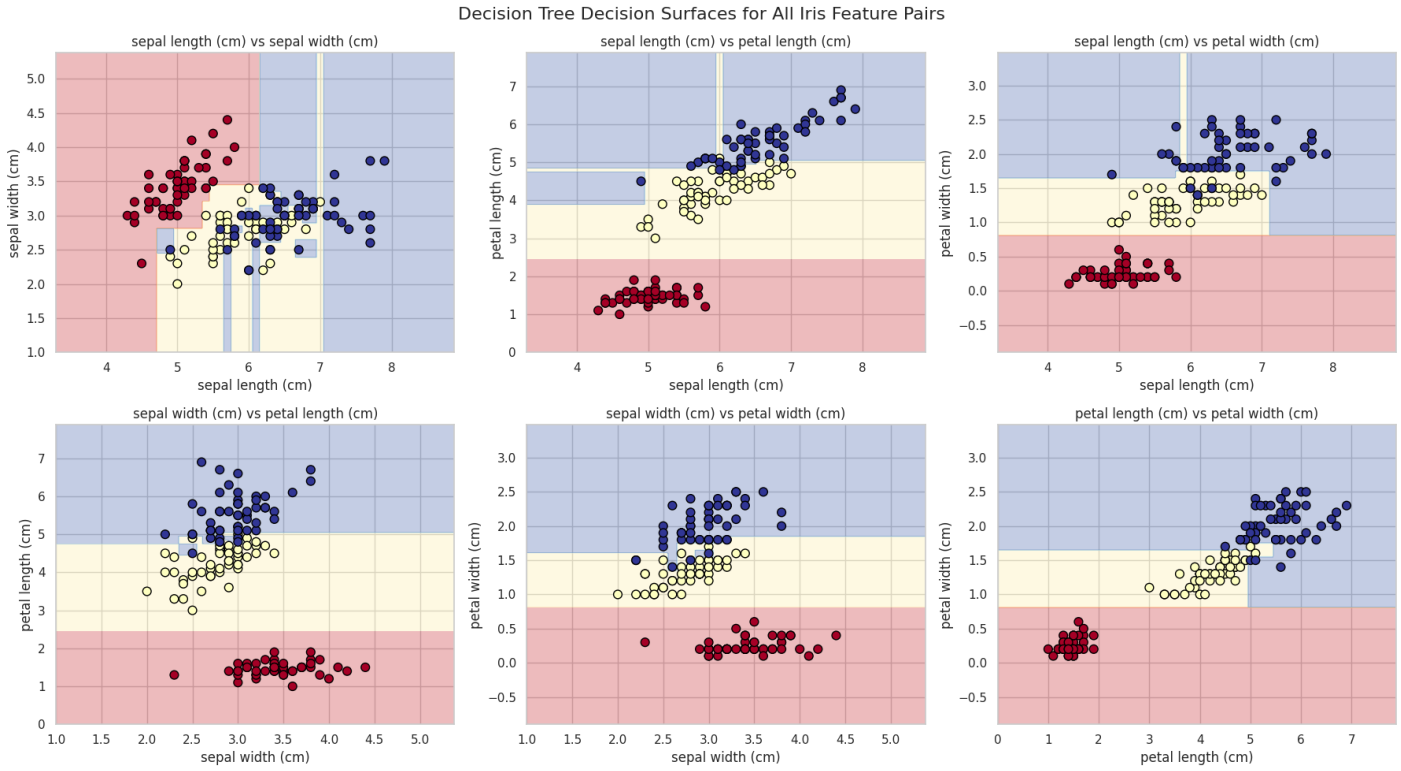



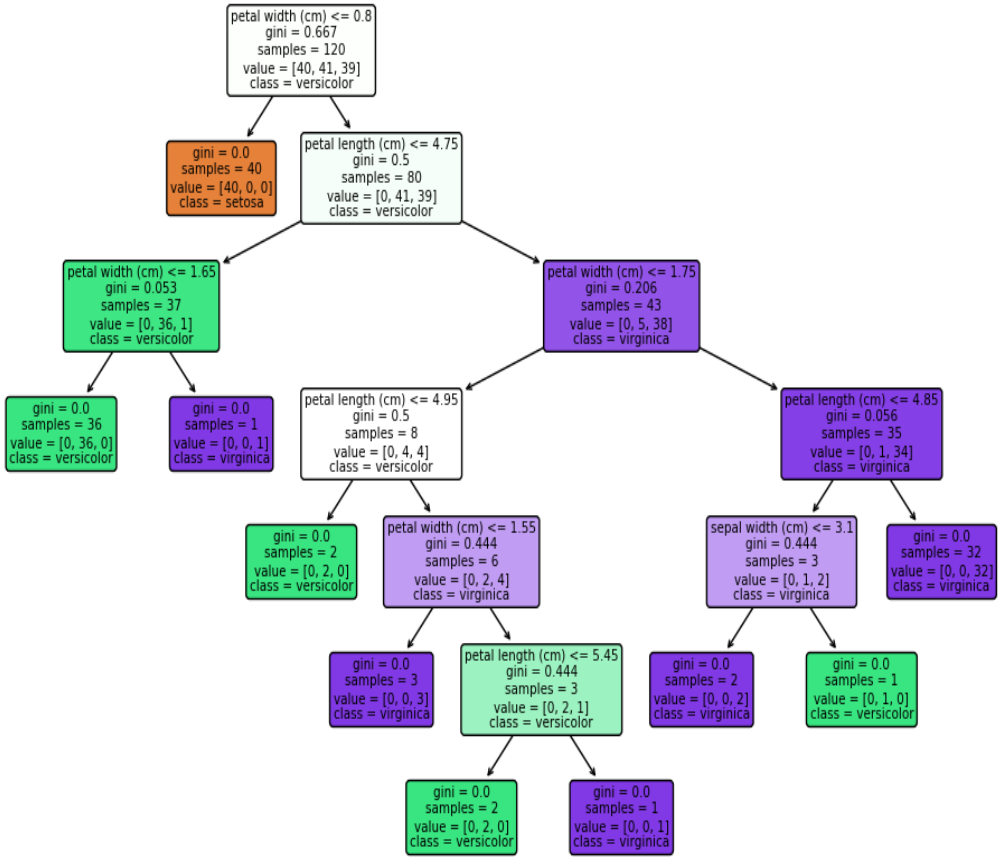
# Decision tree



# Decision tree



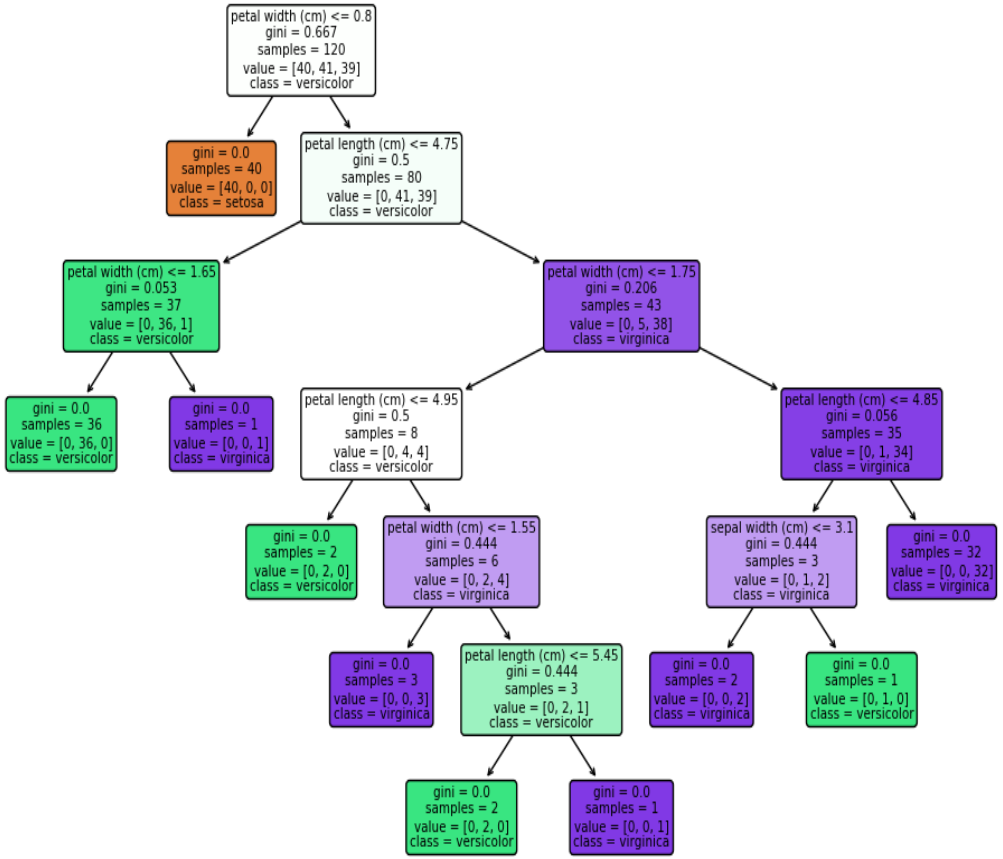
# Decision tree

## Pro's

- Simple to understand (**white box**)
- Require little data preprocessing
- Automatically handles non-linear relationships
- **Computationally efficient**
- Flexible output (classification, regression, multi-class)

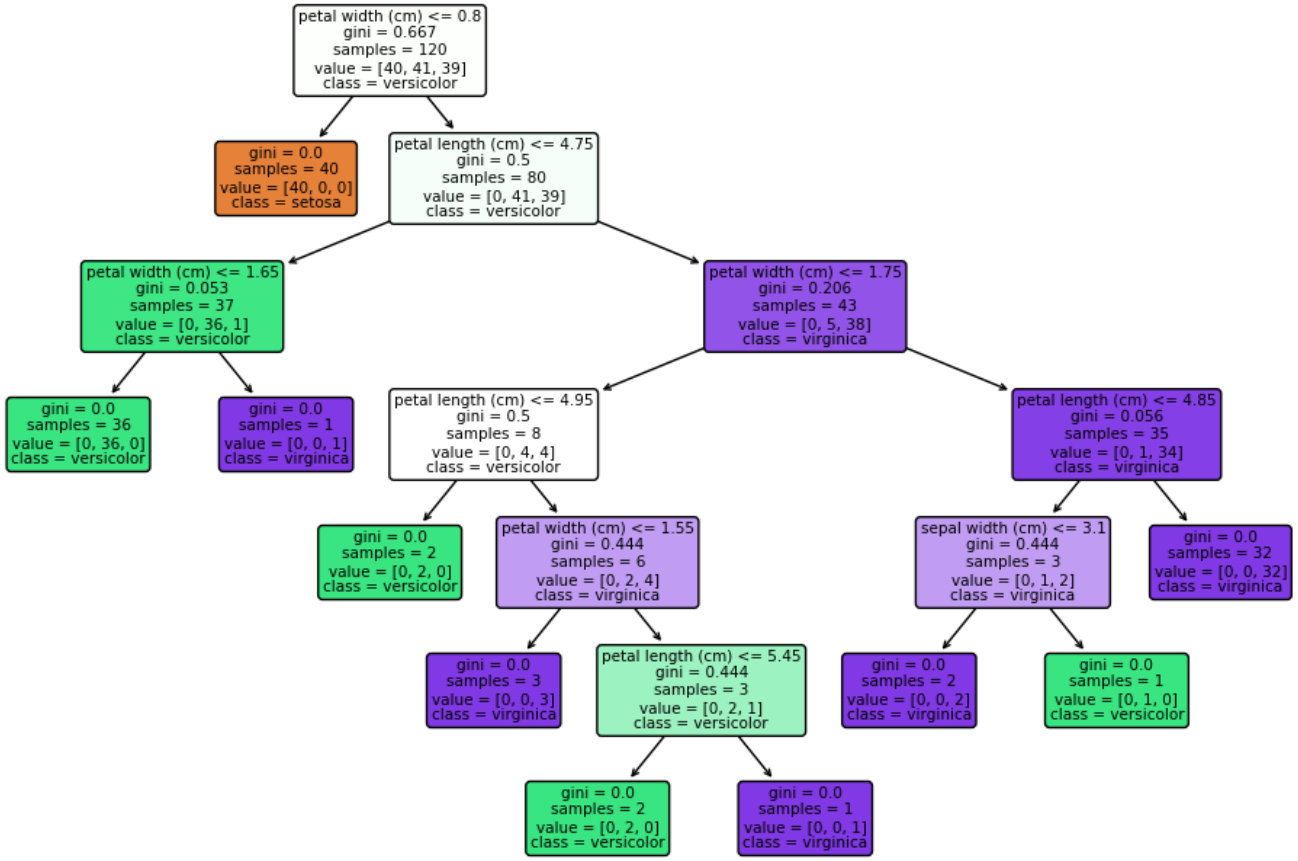
## Cons

- **High risk of overfitting** (greedy)
- Large trees become hard to interpret
- No extrapolation (outliers!)

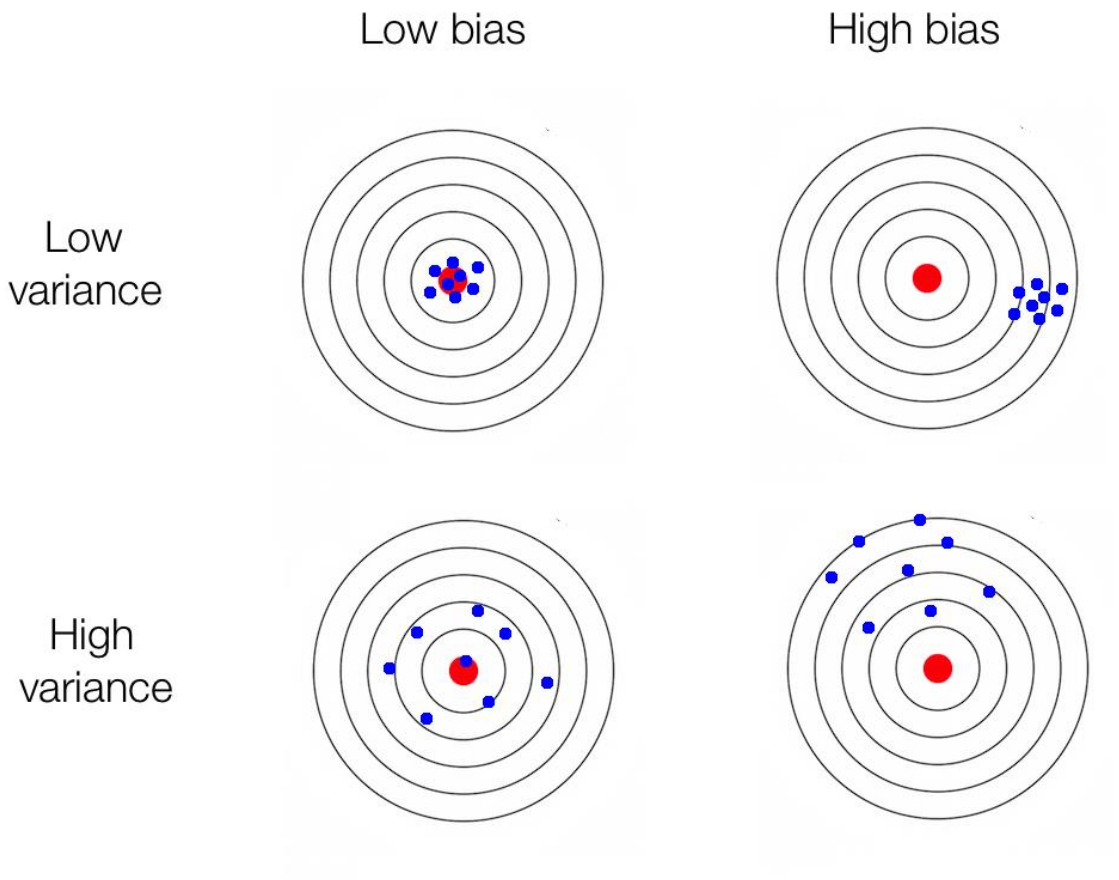


# Decision tree

- **Early stopping**
  - Limit the tree depth
  - Minimum samples in leaves
  - Minimum impurity decrease
- **Pruning**
  - cut back branches that do not improve generalization
- **Ensemble methods**
  - Bagging (RandomForest)
  - Boosting (AdaBoost)



# Ensemble methods



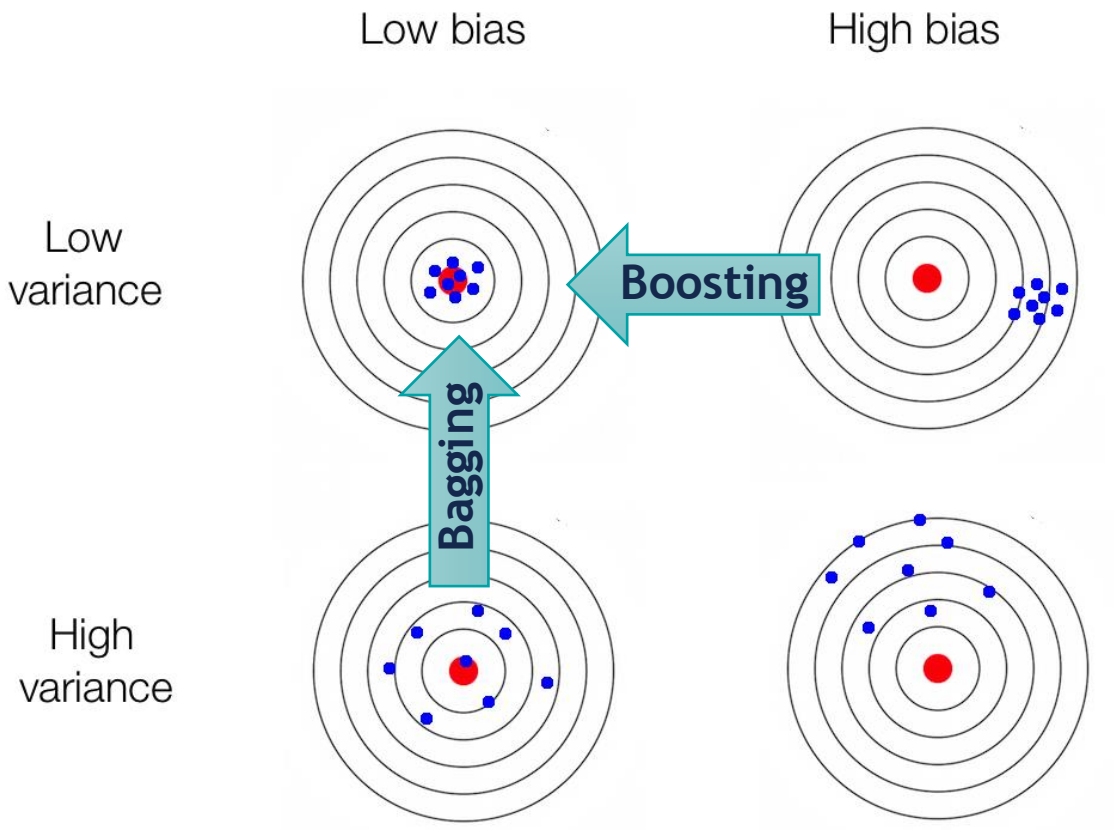

**Bias**

error introduced by approximating a complex problem with a simple model (underfitting)

**Variance**

model's sensitivity to small fluctuations in the training data (overfitting)

# Ensemble methods




## Boosting

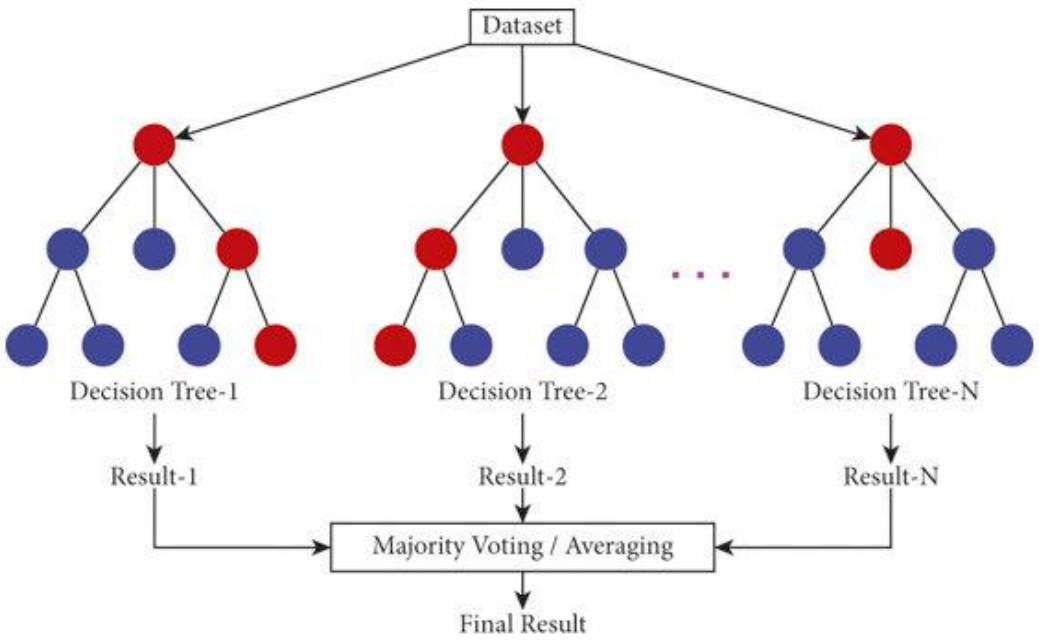
- sequentially train models
- each model corrects the errors of the previous one

## Bagging

- train multiple (small) models
- combine their predictions (averaging/majority voting)

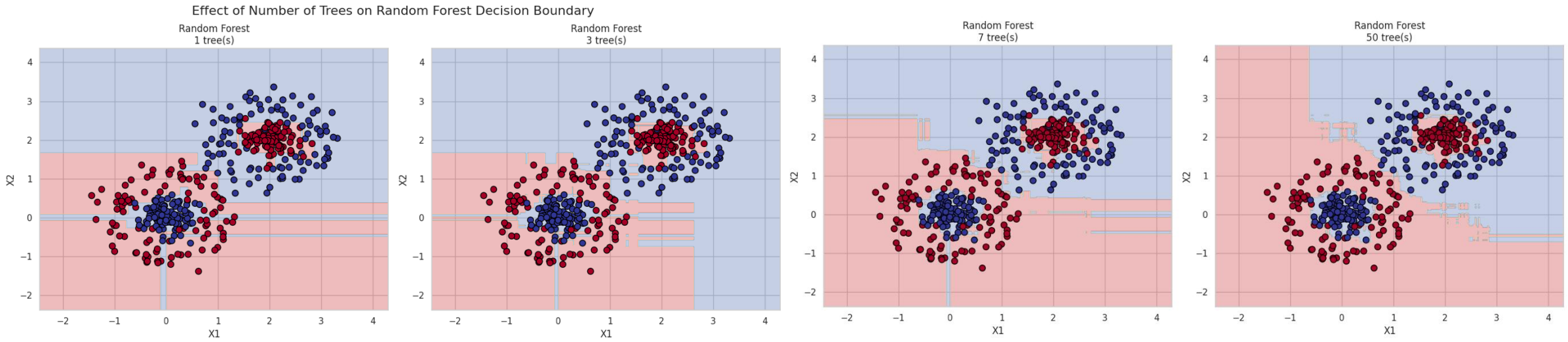
# Random forest

- Train T decision trees with low bias and high variance
  1. Select a subset of the data samples
  2. Select a subset of the features
  3. Grow a (full) tree
- Combine the predictions (classification: majority voting; regression: averaging)



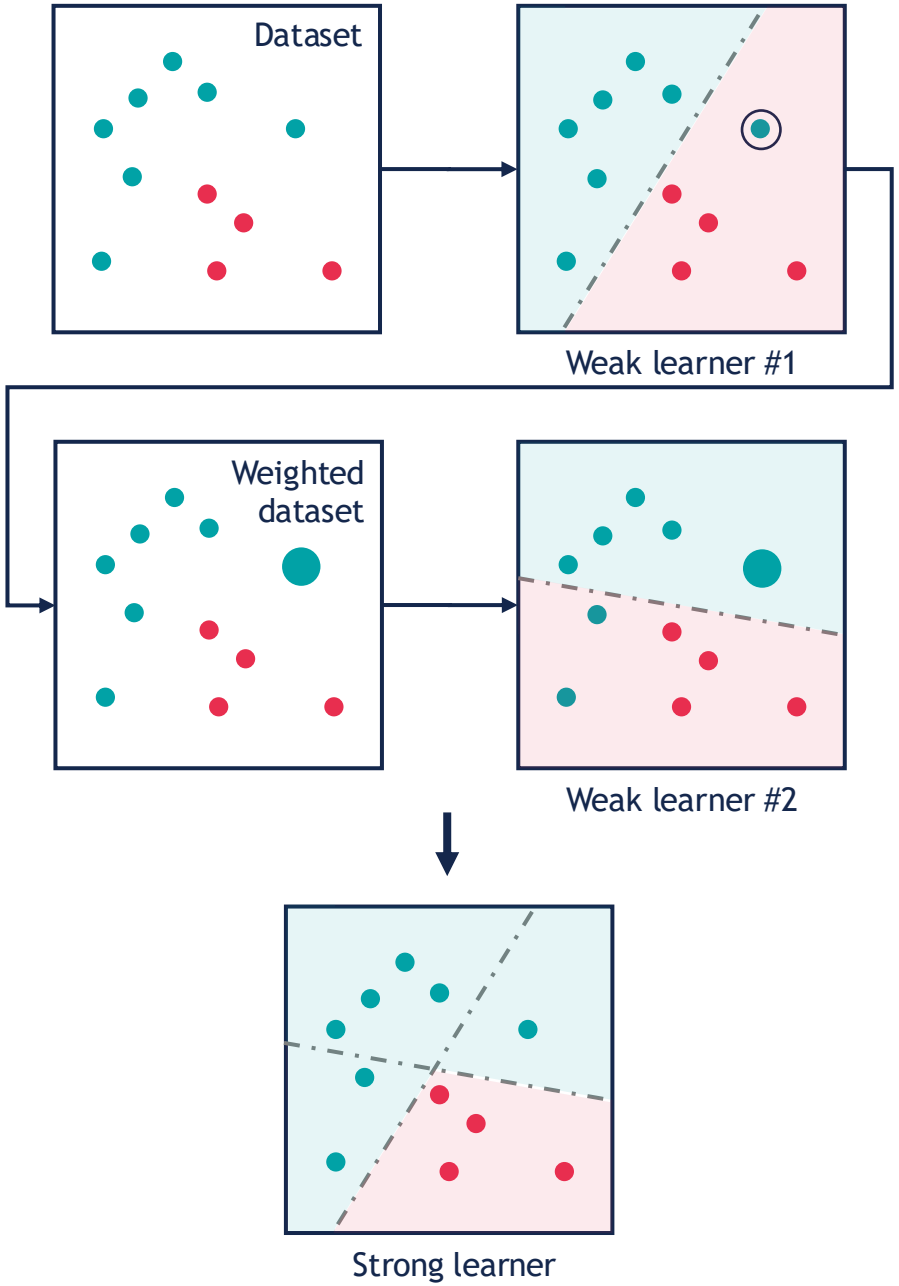

Automated Prediction of Good Dictionary EXamples (GDEX): A Comprehensive Experiment with Distant Supervision, Machine Learning, and Word Embedding-Based Deep Learning Techniques - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/Illustration-of-random-forest-trees_fig4_354354484 [accessed 8 Dec 2025]

# Random forest

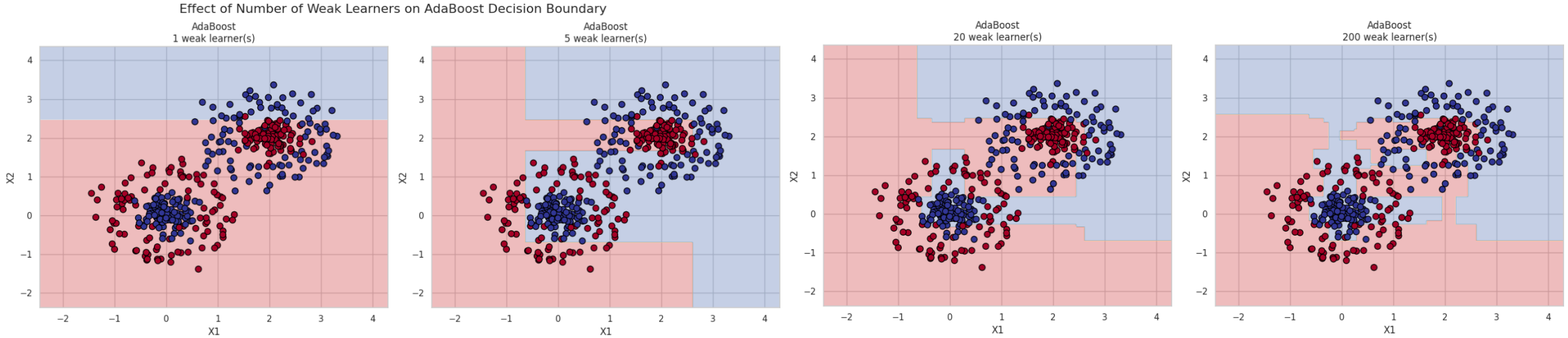


# AdaBoost

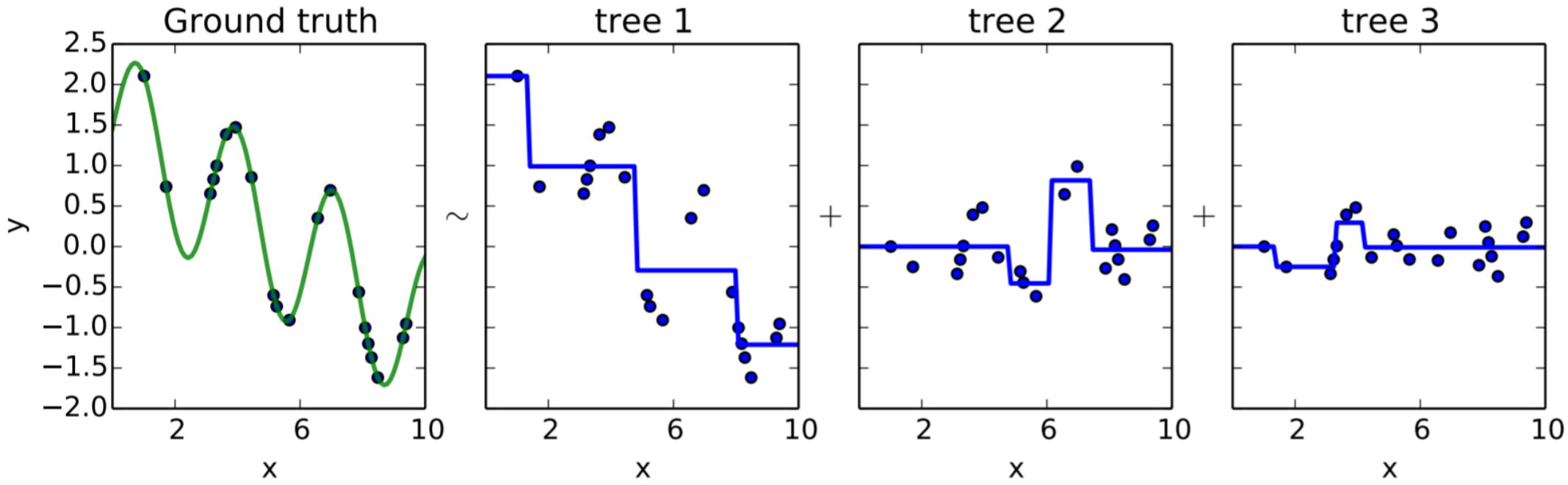
- Train multiple weak learners (e.g. small decision trees) that focus more on the mistakes of the previous classifiers
  1. Train a weak learner
  2. Measure its errors (misclassified → higher weight)
  3. Train next weak learner on weighted data
- Final prediction: weighted sum of the predictions (good weak learners → higher weight)



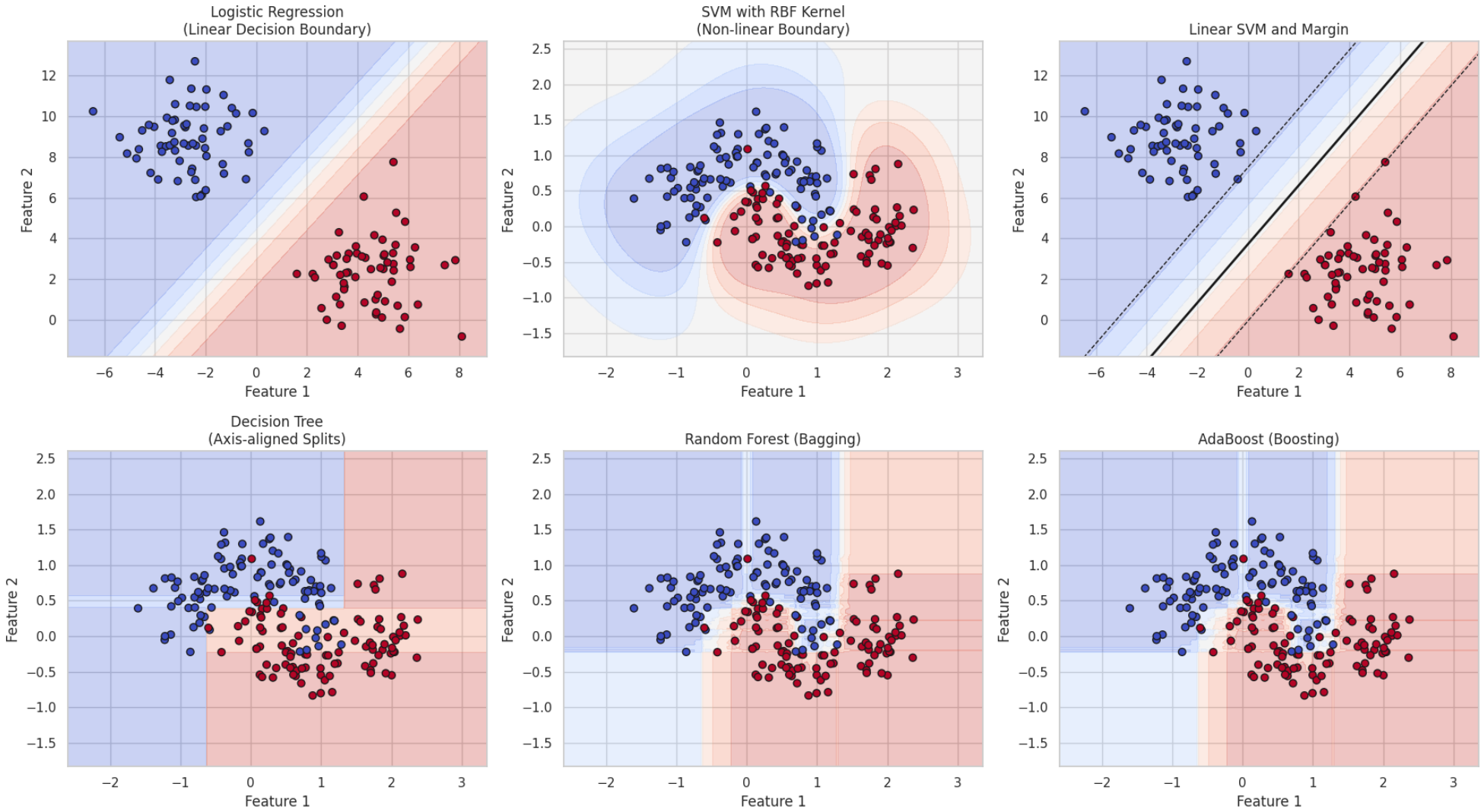
# AdaBoost


Effect of Number of Weak Learners on AdaBoost Decision Boundary
AdaBoost
1 weak learner(s)
AdaBoost
5 weak learner(s)
AdaBoost
20 weak learner(s)
AdaBoost
200 weak learner(s)
X1
X2

# Gradient boosting



# Classification - summary



# Classifiers overview

| | Logistic Regression | SVM | Decision Tree | Random Forest |
|---|---|---|---|---|
| Training | | | | |
| Data requirements | | | | |
| Predictions | | | | |
| Generalization | | | | |

VIB

# Classifiers overview

| | Logistic Regression | SVM | Decision Tree | Random Forest |
|---|---|---|---|---|
| **Training** | Very fast; excellent scalability | | | |
| **Data requirements** | Scaled numerical features | | | |
| **Predictions** | Probabilistic and interpretable | | | |
| **Generalization** | Good with regularization | | | |

VIB

# Classifiers overview

| | Logistic Regression | SVM | Decision Tree | Random Forest |
|---|---|---|---|---|
| **Training** | Very fast; excellent scalability | Slower, especially with kernels | | |
| **Data requirements** | Scaled numerical features | Scaled numerical features; handles high-dimensionality and collinearity well | | |
| **Predictions** | Probabilistic and interpretable | Margin-based; probabilities need calibration | | |
| **Generalization** | Good with regularization | Good with proper C/Kernel | | |

VIB

# Classifiers overview

| | Logistic Regression | SVM | Decision Tree | Random Forest |
|---|---|---|---|---|
| **Training** | Very fast; excellent scalability | Slower, especially with kernels | Very fast, minimal tuning | |
| **Data requirements** | Scaled numerical features | Scaled numerical features; handles high-dimensionality and collinearity well | No scaling needed; handles mixed data types | |
| **Predictions** | Probabilistic and interpretable | Margin-based; probabilities need calibration | Rule-based, interpretable | |
| **Generalization** | Good with regularization | Good with proper C/Kernel | High risk of overfitting | |

VIB

# Classifiers overview

| | Logistic Regression | SVM | Decision Tree | Random Forest |
|---|---|---|---|---|
| **Training** | Very fast; excellent scalability | Slower, especially with kernels | Very fast, minimal tuning | Moderately slow |
| **Data requirements** | Scaled numerical features | Scaled numerical features; handles high-dimensionality and collinearity well | No scaling needed; handles mixed data types | No scaling needed; handles mixed data types |
| **Predictions** | Probabilistic and interpretable | Margin-based; probabilities need calibration | Rule-based, interpretable | Smooth aggregated predictions |
| **Generalization** | Good with regularization | Good with proper C/Kernel | High risk of overfitting | Very good due to averaging |

VIB